

Технічний опис застосунку та план ручного QA-тестування

Як застосунок працює, як він канонічно спроектований, що вже реалізовано, а що ще ні — база для мануального тестування (trainee/junior QA).

Версія документа станом на 31.08.2026, гілка `main`, коміт `0f0a858`

Аудиторія QA (trainee/junior), продакт/розробник як довідка

Джерело істини код репозиторію + ARCHITECTURE.md, docs/*.md, README.md

Документ побудований на прямому огляді коду (роутів API, сторінок фронтенду, доменної логіки), а не лише на документації репозиторію — там, де документація описує щось, чого в коді ще немає, це явно позначено нижче.

1. Про застосунок

Job Intelligence Platform — персональний пошуковий рушій вакансій. Застосунок сам збирає вакансії з кількох джерел (DOU, Djinni), приводить їх до єдиного формату, оцінює кожну відносно структурованого профілю кандидата, пояснює кожну оцінку (чому саме такий відсоток) і надсилає релевантні вакансії користувачу в Telegram.

Це не просто агрегатор списків — ключова цінність продукту в трьох речах:

- **Матчинг, який можна пояснити.** Жодна оцінка не є "чорною скринькою" — завжди видно розбивку по компонентах, сильні сторони та прогалини.
- **Дедуплікація між джерелами.** Та сама вакансія, опублікована на DOU і на Djinni, показується як одна позиція.
- **Проактивна доставка.** Не потрібно щодня заходити й переглядати списки — відповідні вакансії самі приходять у Telegram.

Застосунок — модульний моноліт: один backend (FastAPI) + один пул фонових воркерів (Celery), організований по доменним модулям, і окремий frontend (React SPA). Це особисто-масштабний ("personal-scale") проєкт — реєстрація відкрита, без підтвердження e-mail і без відновлення пароля, розрахований на невелику кількість користувачів (друзі/сам розробник), а не на публічний масовий сервіс.

2. Архітектура та принципи

2.1 Огляд системи



2.2 Конвеєр обробки вакансії (data pipeline)



Кожна стрілка — окрема, незалежно повторювана фонові задача Celery. Одна вакансія проходить: **Raw** (як є від джерела) → **Normalized** (уніфікована форма) → **Canonical** (дедуплікована, одна реальна вакансія, можливо з кількох джерел одночасно).

2.3 Технологічний стек

Шар	Рішення
Frontend	React, TypeScript, Vite, TanStack Query, React Router, Tailwind CSS
Backend	Python 3.13+, FastAPI, Pydantic, SQLAlchemy 2, Alembic
База даних	PostgreSQL + pgvector
Черга	Redis + Celery (Celery Beat для розкладу)
Скrapінг	httpx, BeautifulSoup/lxml; Playwright — лише як fallback для JS-залежних сторінок
Embeddings	sentence-transformers (локально, за замовчуванням) або OpenAI
LLM	Провайдер-абстракція: Ollama (локально, за замовчуванням), OpenAI, Anthropic, Gemini (опційно, тільки для аналізу CV)
Нотифікації	Telegram Bot API (єдиний реалізований канал)

Інфраструктура	Docker, Docker Compose; прод — Cloudflare Pages (frontend) + Oracle Cloud VM (backend)
Автотести	pytest (backend, unit) — див. розділ 10 щодо відсутності автотестів фронтенду

2.4 Архітектурні принципи ("канонічні" правила проєкту)

Це не побажання, а обмеження, яких мав дотримуватись кожен модуль. Знання їх допомагає розуміти, чому застосунок поводить ся саме так, а не інакше:

1. **Адаптери, а не спецвипадки.** Джерела вакансій, LLM-провайдери, провайдери ембедингів і канали нотифікацій — взаємозамінні адаптери за вузькими інтерфейсами.
2. **Детермінізм — насамперед.** Оцінка вакансії — ніколи не "спитати LLM який відсоток". Спершу жорсткі фільтри й зважений детермінований скоринг; embeddings додають нюанс; LLM переранжує лише невелику вже відфільтровану вершину списку.
3. **Пояснюваність обов'язкова.** Кожна оцінка супроводжується розбивкою по компонентах, сильними сторонами й прогалинами. Голе число — це баг.
4. **Досвід ≠ побажання.** Що кандидат уміє (`CandidateProfile` , з CV) — окрема модель від того, чого кандидат хоче (`UserPreference` , редагується вручну).
5. **Raw ≠ Normalized ≠ Canonical.** Див. п. 2.2.
6. **Ідемпотентність фонових задач.** Скрапінг, нормалізація, скоринг, доставка нотифікацій — усе безпечно повторити без дублювання.
7. **Відмова одного джерела — не відмова платформи.** Якщо Djinni зламався, DOU й решта застосунку продовжують працювати.
8. **Модульний моноліт, не мікросервіси** — доки конкретна проблема масштабування цього не вимагатиме.
9. **LLM — інструмент міркування, не джерело істини.** LLM ніколи не визначає ID дедуплікації, статус вакансії, дати, стан нотифікацій/заявок чи вподобання користувача.

3. Ключові поняття (глосарій)

Raw / Normalized / Canonical Job

RawJob — точно те, що адаптер отримав із джерела (сире HTML/JSON). **NormalizedJob** — уніфікована форма (назва, компанія, опис, зарплата, локація, seniority, навички), незалежна від джерела. **CanonicalJob** — дедуплікована реальна вакансія, за якою може стояти декілька `JobSourceRecord` (та сама вакансія на DOU і на Djinni одночасно — тоді на бейджі в UI буде "2 sources").

CandidateProfile vs UserPreference

CandidateProfile — що кандидат реально вміє (досвід, ролі, навички, історія роботи) — витягується з CV через LLM, редагується опосередковано (перезавантаженням/переаналізом CV). **UserPreference** — чого кандидат хоче (бажана зарплата, локації, формат роботи, чорні списки) — редагується напряму на сторінці Settings. *CandidateProfile* відповідає на "чи можу я це робити?", *UserPreference* — на "чи хочу я цю роботу?".

Requirement Match vs Practical Fit

Дві окремі оцінки, які показуються поруч, а не змішуються в одне число: **Requirement Match** — наскільки буквально CV задовольняє перелічені вимоги; **Practical Fit** — наскільки добре кандидат реально впорається, враховуючи трансферні навички (наприклад, Django-досвід рахується частково релевантним для вакансії на FastAPI).

Recommendation: APPLY / CONSIDER / SKIP

Результат детермінованого скорингу. `APPLY` — вакансія, яку варто розглянути першочергово (і саме для таких вакансій, у межах денного ліміту викликів, додатково запускається LLM-аналіз "Should I apply?" — див. 5.7). Список `Jobs` за замовчуванням приховує `SKIP`-вакансії (чекбокс "Show all, including low matches" показує їх).

Жорсткі фільтри (hard filters)

Дешеві детерміновані правила прийняти/відхилити, що виконуються ще до скорингу: релокація в країну, куди кандидат не готовий переїжджати; зарплатна стеля нижче мінімуму кандидата; вимога допуску/громадянства; необхідний досвід far поза межами кандидата; заблокований стек. Вакансія, що не пройшла фільтр, позначається `eligible: false` і не сповіщається.

Джерела: DOU і Djinni

DOU — виявлення нових вакансій через RSS-фід (стабільніше за парсинг HTML-списку), деталі — з HTML-сторінки вакансії. Djinni — HTML через `httpx/BeautifulSoup`, `Playwright` лише як крайній fallback. Кожне джерело сканується по одній категорії за раз (ротація категорій, розділ 2.2), свій "health" відстежується незалежно від іншого джерела.

LLM-провайдери й політика вибору

Дві незалежні точки виклику LLM: **аналіз CV** (рідко, вручну користувачем — якість найважливіша: якщо задано `GEMINI_API_KEY`, спершу пробує безкоштовний Gemini, переходить на Ollama лише при 429/вичерпаній квоті, на будь-якій іншій помилці — падає з помилкою, а не мовчки деградує) і **вилучення навичок вакансії** (на кожную нову вакансію, завжди Ollama — щоб не витратити обмежену безкоштовну квоту Gemini). У відповіді завжди видно, яка модель реально відповіла (`model_label` / "Analyzed using ..." / "Assessed using ...").

Ротація категорій і retention

Кожен тик скрапінгу покриває **одну** категорію джерела (не весь фід одразу) — вибирається та, яку скановано найдавніше. Раз на добу видаляються вакансії (і все, що на них посилається: матчі, нотифікації, заявки), не бачені в скрапінгу довше `JOB_RETENTION_DAYS` (18 днів за замовчуванням) — щоб база не росла необмежено.

4. Легенда статусів реалізації

У розділі 5 та в матриці API кожна можливість позначена одним із трьох статусів:

 **РЕАЛІЗОВАНО** — працює на реальних даних, перевірено, можна повноцінно тестувати

 **ЧАСТКОВО** — базова версія працює, але описана в документації поведінка урізана або відсутня частина

 **НЕ РЕАЛІЗОВАНО** — заглушка/500-помилка; тестувати нема чого, лише перевірити, що це саме заглушка, а не крах

Важливо для QA: усе, позначене  або  у цьому документі — це відоме, задокументоване обмеження поточної версії, а не знахідка. Заводити баг-репорт на нього не потрібно (хіба що поведінка відрізняється від опису тут — наприклад, замість "заклушки" сторінка падає з білим екраном). Розділ 8 містить зведений список.

5. Функціонал за екранами

Навігація (header) містить лише: Dashboard, Jobs, Profile, Settings, Sources. Applications і Market Insights у навігації немає — вони існують лише за прямим переходом за URL.

5.1 Реєстрація та вхід — ✓ РЕАЛІЗОВАНО

Маршрути: `/register`, `/login`. **API:** `POST /api/auth/register`, `POST /api/auth/login`, `GET /api/auth/me`.

Канонічна поведінка: відкрита самореєстрація за email + паролем (мінімум 8 символів), без підтвердження пошти й без відновлення пароля — свідоме спрощення для невеликого особистого застосунку. Токен зберігається в `localStorage`; усі внутрішні маршрути захищені компонентом `RequireAuth` і без токена редіректять на `/login`.

Що перевіряти: успішна реєстрація/логін, дубль email → 409, невірний пароль → 401, доступ без токена → редірект, `logout` з хедера очищує токен.

5.2 Dashboard — ✓ РЕАЛІЗОВАНО

Маршрут: `/`. Показує чек-лист налаштування (CV завантажено / переваги задано / вакансії заскрапано / Telegram підключено) з посиланнями на відповідні сторінки, і короткий блок "At a glance" (кількість канонічних вакансій, сирих вакансій, CV). Коли всі 4 пункти виконані — з'являється CTA "Score jobs →".

Що перевіряти: коректність позначок ✓/○ відповідно до реального стану даних; коректність лічильників; всі посилання ведуть куди слід.

5.3 Profile — завантаження й аналіз CV — ✓ РЕАЛІЗОВАНО

Маршрут: `/profile`. **API:** `POST/GET /api/cv`, `POST /api/cv/analyze`, `GET /api/cv/profile`.

Приймає `.txt`, `.pdf`, `.docx`. Кнопка "Analyze most recent CV" запускає LLM-екстракцію структурованого `CandidateProfile` (досвід у роках, ролі, навички з рівнем, досягнення, домени) з поточного останнього CV. Після переаналізу в фоні автоматично перезапущається скоринг **усіх** вже заскрапаних вакансій відносно нового профілю (не лише майбутніх) — у самому UI це не візуалізовано, ефект видно пізніше на сторінці Jobs/Job Details.

Що перевіряти: завантаження всіх 3 форматів, помилка на невідтримуваному форматі (422), відображення профілю після аналізу, підпис "Analyzed using ..." (яка модель реально відповіла).

5.4 Settings — Preferences та Telegram — ✓ РЕАЛІЗОВАНО ⚠ частково щодо документації

Маршрут: `/settings`. **API:** `GET/PATCH /api/settings`, `GET /api/integrations/telegram/*`.

Preferences: зарплата, макс. потрібний досвід, бажані ролі, локації, формати роботи, стек preferred/acceptable/blocked, чорні списки компаній/індустрій — усе редагується й одразу зберігається. **Telegram:** показ username спільного бота (токен — на сервері, per-user потрібен лише `chat_id`), кнопки Connect і Send test message.

Розбіжність з документацією: `docs/api.md` описує сторінку Settings як таку, що містить ще й "matching weights" (ручне налаштування ваг компонентів скорингу) — у поточному UI такого блоку немає. Не шукати й не заводити баг — ваги скорингу наразі не редагуються користувачем.

Що перевіряти: збереження й персистентність кожного поля переваг; підключення Telegram валідним/невалідним `chat_id`; тестове повідомлення реально приходить у Telegram.

5.5 Sources — ✓ РЕАЛІЗОВАНО ⚠ частково щодо документації

Маршрут: `/sources`. **API:** `GET /api/sources`, `POST /api/sources/{id}/sync`.

Список джерел (`dou`, `djinni`) з кількістю збережених сирих вакансій і кнопкою "Sync now", яка ставить у чергу позачерговий скрапінг.

Розбіжність з документацією: `docs/api.md` описує цю сторінку як таку, що показує "per-source health (ScrapeRun history, degraded/healthy)". У поточному UI цього немає — видно лише назву джерела й лічильник. Здоров'я джерела бекенд відстежує внутрішньо (`ScrapeRun` , лічильник послідовних помилок), але на фронтенд це не виведено. Якщо потрібно перевірити, чи джерело деградувало — це можна побачити лише через БД/логи бекенду, не через UI.

5.6 Jobs — список вакансій — ✓ РЕАЛІЗОВАНО

Маршрут: `/jobs`. **API:** `GET /api/jobs ( ?limit&offset&include_skipped )`.

Пагінація по 25. За замовчуванням приховує вакансії з рекомендацією `SKIP` (чекбокс "Show all, including low matches" показує все). Бейдж `practical_fit%` для оцінених вакансій, бейдж "N sources" для вакансій з кількох джерел. Кнопка "Score unscored on this page" масово ставить у чергу неоцінені вакансії поточної сторінки.

Немає (і це не баг цієї версії): фільтрів за назвою/компанією/навичкою/зарплатою, сортування, пошуку — мінімальний UI v1.

5.7 Job Details — деталі й пояснення оцінки — ✓ РЕАЛІЗОВАНО

Маршрут: `/jobs/:jobId`. **API:** `GET /api/jobs/{id}`, `GET /api/jobs/{id}/match`, `POST /api/jobs/{id}/rescore`.

Опис вакансії + картка "Match": Practical fit, Requirement match, бейдж Recommendation, розбивка по 8 компонентах (`skills`, `role`, `experience`, `semantic_fit`, `salary`, `location`, `transferable_skills`, `preferences` — кожен у %), чипи `strengths` (зелені) і `gaps` (жовті), підпис "Skills identified using ...", кнопка Rescore. Якщо вакансія ще не оцінена — замість цього кнопка "Score this job".

Картка "Should I apply?" — умовна, показується не завжди

Ця картка з'являється **лише** тоді, коли: (а) детермінований конвеєр вже рекомендує `APPLY`, і (б) не вичерпано денний ліміт LLM-викликів (`LLM_RERANK_DAILY_LIMIT`). Тобто на `CONSIDER` / `SKIP` -вакансії або на `APPLY` -вакансії пізно ввечері після вичерпання ліміту картки не буде — **це очікувано, не**

баг. Коли показана: recommendation, confidence %, interview risk, короткий summary, critical gaps (червоні чипи), transferable experience (зелені чипи), опційно recommended CV-варіант, підпис "Assessed using " (може виявитись Ollama-фолбек, якщо основний провайдер вичерпав квоту — теж очікувано).

На цій сторінці немає кнопок Save / Apply / Reject / Hide — ці дії не реалізовані (див. розділ 8, "Application-трекер"). Не шукати їх і не заводити баг на відсутність.

5.8 Applications — ✗ НЕ РЕАЛІЗОВАНО

Маршрут: `/applications` (лише за прямим URL, немає в навігації). Фронтенд — буквально заглушка (`<h1>Applications</h1>`), без жодних даних чи дій. **API** `GET /api/applications` повертає 500 (`NotImplementedError`). Це запланований трекер заявок (discovered → applied → interview → offer/rejected) — Phase 5, ще не почато на рівні бізнес-логіки.

Що перевіряти: лише що сторінка відкривається без критичного крашу застосунку (білий екран/ незловлений виняток) — сам факт "порожньо" очікуваний.

5.9 Market Insights — ✗ НЕ РЕАЛІЗОВАНО

Маршрут: `/market-insights` (лише за прямим URL). Так само буквальна заглушка (`<h1>Market Insights</h1>`). Заплановано: аналітика попиту на навички, "які навички відкривають більше вакансій" — Phase 5.

6. Наскрізний сценарій (happy path)

Базовий сценарій, який варто пройти цілком хоча б раз — він зачіпає майже весь реалізований функціонал і найкраще виявляє інтеграційні проблеми (а не лише проблеми окремого екрана).

1. Зареєструватися новим email (`/register`) → автоматичний вхід, редірект на Dashboard.
2. На Dashboard переконатись, що всі 4 пункти чек-листа не позначені.
3. Profile → завантажити CV (.pdf/.docx/.txt) → "Analyze most recent CV" → дочекатись появи картки CandidateProfile.
4. Settings → заповнити Preferences (мінімум: бажана зарплата, локації, preferred stack) → зберегти.
5. Settings → написати спільному Telegram-боту (посилання на сторінці), отримати свій `chat_id` , ввести його й натиснути Connect → статус "Connected".
6. Settings → "Send test message" → перевірити прихід тестового повідомлення в Telegram (87%-приклад).
7. Sources → "Sync now" для dou і для djinni → зачекати (реальний скрапінг триває від десятків секунд до кількох хвилин) → лічильники raw_jobs_stored ростуть.
8. Dashboard → усі 4 пункти чек-листа тепер позначені ✓, з'являється СТА "Score jobs →".
9. Jobs → відкрити список, натиснути "Score unscored on this page" (якщо потрібно), зачекати, оновити сторінку → з'являються бейджі practical_fit%.
10. Відкрити вакансію з високим practical_fit → Job Details → перевірити розбивку скорингу, strengths/gaps; якщо рекомендація APPLY — перевірити наявність (або обґрунтовану відсутність) картки "Should I apply?".
11. Якщо якась вакансія набрала $\geq 85\%$ і сталось це поза тихими годинами (08:00–22:00 за часом сервера) — перевірити, що прийшло реальне сповіщення в Telegram з правильним форматом і робочими (нефункціональними по кліку — це очікувано) inline-кнопками.
12. Logout → переконатись, що повторний прямий перехід на `/jobs` редіректить на `/login` .

7. Матриця API endpoints

Для довідки / для тих, хто тестуватиме і на рівні API (Postman, curl), не лише через UI. "Використовує" — яка сторінка фронтенду реально викликає цей endpoint.

Метод і шлях	Статус	Використовує / примітка
POST /api/auth/register	✓	Register
POST /api/auth/login	✓	Login
GET /api/auth/me	✓	—
POST /api/cv	✓	Profile — завантаження CV
GET /api/cv	✓	Profile — список CV
POST /api/cv/analyze	✓	Profile — Analyze
GET /api/cv/profile	✓	Profile — картка профілю
GET /api/profile	✓	Dashboard — зведення (лише для читання)
GET /api/jobs	✓	Jobs
GET /api/jobs/{id}	✓	Job Details
GET /api/jobs/{id}/match	✓	Job Details
POST /api/jobs/{id}/rescore	✓	Jobs, Job Details
POST /api/jobs/{id}/save	✗ 500	Не викликається жодним екраном (Phase 5)
POST /api/jobs/{id}/apply	✗ 500	Не викликається (Phase 5)
POST /api/jobs/{id}/reject	✗ 500	Не викликається (Phase 5)
GET /api/matches	✗ 500	Не викликається жодним екраном
GET /api/sources	✓	Sources, Dashboard
POST /api/sources/{id}/sync	✓	Sources
GET/POST /api/search-profiles	✗ 500	Немає UI взагалі
PATCH /api/search-profiles/{id}	✗ 500	Немає UI взагалі
GET /api/applications	✗ 500	Applications (заглушка)
GET/PATCH /api/settings	✓	Settings — Preferences
GET /api/integrations/telegram/bot-info	✓	Settings
GET /api/integrations/telegram/status	✓	Settings, Dashboard
POST /api/integrations/telegram/connect	✓	Settings
POST /api/integrations/telegram/test	✓	Settings

8. Зведений список: що ще НЕ реалізовано

Повний перелік запланованого, але відсутнього функціоналу — щоб під час тестування було одразу зрозуміло, що це "ще не побудовано", а не "зламано".

Повністю відсутнє (✗)

- **Application-трекер** (сторінка Applications, дії Save/Apply/Reject, статуси discovered→applied→interview→offer/rejected) — лише інтерфейси/моделі даних без бізнес-логіки.
- **Market Insights** (аналітика попиту на навички, "яка навичка відкриє +N% вакансій").
- **Search Profiles** (збережені пошукові профілі) — ні API, ні UI.
- **GET /api/matches** (загальний список матчів поза сторінкою Jobs) — заглушка.
- **Обробка натискань inline-кнопок Telegram** (Apply/Save/Not relevant у повідомленні) — кнопки рендеряться, але натискання нічого не робить на сервері (немає webhook/callback-обробника).
- **Daily digest** — вакансії з оцінкою 65–84%, які не пройшли поріг миттєвого сповіщення, зараз не потрапляють нікуди: не надсилаються одразу і не збираються в зведення пізніше.
- **"Ранкове зведення" після тихих годин** — по факту тихі години (22:00–08:00) просто *гасять* сповіщення, а не відкладають їх; жодного "ранкового" дайджесту немає, попри те, що так описано в `docs/notifications.md`.
- **Batch LLM reranking** (`rerank_shortlist`) над явним шортлистом — немає ні вью шортлиста, ні дайджест-батчингу, який мав би його наповнювати.
- **Ручне налаштування ваг скорингу** через UI (Settings) — ваги компонентів (skills 30%, role 15% тощо) захардкоджені на бекенді.
- **Персистентні embeddings (pgvector)** — обчислюються "на льоту" при кожному скорингу замість збереження в БД; коректно, але не так, як описано в оригінальному дизайні.
- **Feedback → навчання ваг** ("ви відхилили 11/12 Angular-вакансій, зменшити вагу?") — не реалізовано.
- **Кілька CV-варіантів з автоматичним підбором** ("recommended_cv" у Should I apply? може з'явитись у відповіді LLM як текст, але немає UI для керування кількома CV-профілями, крім "останній завантажений CV = поточний профіль").
- **Здоров'я джерел у UI** (ScrapeRun-історія, healthy/degraded) — трекається на бекенді, не виводиться на фронтенд.

Свідомо відсутнє за дизайном (не в планах як "баг", а як рішення)

- Підтвердження email і відновлення пароля.
- Фільтри/пошук/сортування у списку Jobs.
- E2E/автотести фронтенду (Vitest/Playwright не підключені) — див. розділ 10.

9. Технічні нюанси — НЕ репортити як баг

Ці особливості поведінки виглядають підозріло під час тестування, але є очікуваними наслідками дизайну застосунку:

Спостереження	Пояснення
Щойно розгорнутий застосунок показує 0 вакансій довго	Автоматичний скрапінг триває раз на <code>SCRAPE_INTERVAL_SECONDS</code> (30 хв за замовчуванням) на одну категорію джерела. "Sync now" на Sources — найшвидший спосіб отримати дані для тестування.
Після "Sync now" DOU знову бачу мало нових вакансій	Кожен тик покриває лише одну категорію (ротація), не весь фід — повне покриття всіх категорій набирається за багато тиків.
Аналіз CV займає 10–30 секунд і довше	При <code>LLM_PROVIDER=ollama</code> це CPU-інференс локальної моделі — повільно, але безкоштовно; це очікувана затримка для дії, яку користувач виконує рідко.
Після "Score this job"/"Score unscored" оцінка не з'явилась одразу	Скоринг — фоновая Celery-задача. UI сам інвалідує кеш запиту лише через фіксовані 3 секунди; якщо бекенд не встиг — потрібне ручне оновлення сторінки трохи пізніше.
Картки "Should I apply?" немає навіть на вакансії з високим %	Показується лише для <code>APPLY</code> -рекомендацій, і додатково обмежена денним лімітом LLM-викликів (<code>LLM_RERANK_DAILY_LIMIT</code>) — це навмисний контроль вартості.
"Assessed using ..." / "Analyzed using ..." показує не той провайдер, що очікувався	Якщо основний провайдер (Gemini) віддав 429 (квота вичерпана), система автоматично й безшовно перемкнулась на Ollama — і чесно показала це в підписі, а не видала за Gemini.
Стара вакансія раптом зникла зі списку	Retention: вакансії, не бачені в скрапінгу > 18 днів, видаляються разом з матчами й нотифікаціями щоденною фоновною задачею.
Натискання кнопки в Telegram-повідомленні ("Apply"/"Save"/"Not relevant") нічого не робить	Обробка callback-ів не реалізована (див. розділ 8) — кнопки декоративні наразі.
Вакансія з оцінкою 70% ніколи не прийшла в Telegram	65–84% (без сильного збігу по зарплаті/локації для 75–84%) потрапляє в "digest"-діапазон, а daily digest не реалізований — сповіщення для неї не буде взагалі, і це не помилка доставки.

10. Налаштування тестового середовища

У репозиторії наразі **немає автоматизованих тестів фронтенду** (Vitest/Playwright не підключені — лише backend-юніт-тести на pytest, які не потребують БД). Це означає, що ручне QA — єдина перевірка UI застосунку на сьогодні, а не формальність поверх автотестів.

10.1 Локальний запуск (Docker Compose)

- Скопіювати `.env.example` → `.env` у корені репозиторію.
- Мінімально задати: `SECRET_KEY` (будь-який рядок для локального тесту), `TELEGRAM_BOT_TOKEN` і власний `TELEGRAM_CHAT_ID` для перевірки нотифікацій (отримати токен у @BotFather).
- LLM: за замовчуванням `LLM_PROVIDER=ollama` — потрібен локально піднятий Ollama з моделлю (`LLM_MODEL=llama3.2:3b`); або простіше для QA — вказати `ANTHROPIC_API_KEY` / `OPENAI_API_KEY` / `GEMINI_API_KEY` і відповідно змінити `LLM_PROVIDER`, щоб не залежати від локальної Ollama.
- `cd frontend && npm install` (одноразово, потрібно для `docker compose up`).
- `docker compose up` — піднімає Postgres, Redis, API, Celery worker + beat, web.
- Frontend: `http://localhost:5173`. Backend: `http://localhost:8000` (Swagger UI на `/docs` — зручно для тестування API напряму).

10.2 Лише backend (без Docker, для дрібних перевірок API)

```
cd backend
python -m venv .venv
.venv/Scripts/activate
pip install -e ".[dev]"
pytest # unit-тести, без БД
# Для реального запуску API потрібні Postgres + Redis (docker compose up -d postgres redis)
alembic upgrade head
uvicorn app.main:app --reload
```

10.3 Порти й сервіси

Сервіс	Порт	Призначення
web (frontend)	5173	React SPA (Vite dev server)
api	8000	FastAPI, <code>/docs</code> — Swagger
postgres	5432	Основна БД
redis	6379	Черга Celery + кеш

10.4 Що потрібно мати заздалегідь

- Тестовий Telegram-акаунт і власний `chat_id` (написати боту й отримати з відповіді, або через `@userinfobot`).

- Тестовий CV у форматі .pdf/.docx/.txt (реальний або навчальний — головне, щоб містив структуровану інформацію: досвід, навички, посади).
- API-ключ хоча б одного LLM-провайдера, якщо локальна Ollama недоступна/повільна.

11. Чек-лист тест-кейсів

Пріоритет: **P1** — критичний шлях (мати покритим завжди), **P2** — важливо, **P3** — другорядне/крайовий випадок.

Автентифікація (AUTH)

ID	Кроки	Очікуваний результат	Пріор.
AUTH-01	Реєстрація з валідним email і паролем ≥8 символів	201, токен збережено, редірект на Dashboard	P1
AUTH-02	Реєстрація на email, що вже зареєстрований	Помилка "email already registered", форма не відправлена	P1
AUTH-03	Реєстрація з паролем <8 символів	Браузерна валідація блокує сабміт (HTML5 minlength)	P3
AUTH-04	Логін з правильними даними	Успішний вхід, редірект на Dashboard	P1
AUTH-05	Логін з неправильним паролем/email	"invalid email or password" під формою	P1
AUTH-06	Прямий перехід на /jobs без токена (напр. очистити localStorage)	Редірект на /login	P1
AUTH-07	Logout з хедера	Токен видалено, редірект на /login , повторний доступ неможливий без нового логіну	P1

Dashboard (DASH)

ID	Кроки	Очікуваний результат	Пріор.
DASH-01	Відкрити Dashboard одразу після реєстрації	Усі 4 пункти чек-листа не позначені, лічильники 0/—	P1
DASH-02	Поступово виконати CV/preferences/sync/telegram	Відповідний рядок позначається ✓ по мірі виконання	P1
DASH-03	Виконати всі 4 пункти	З'являється блок "Everything's set up" з кнопкою "Score jobs →"	P2
DASH-04	Клікнути кожне посилання чек-листа	Веде на відповідну сторінку (/profile, /settings, /sources)	P2
DASH-05	Звірити лічильники "canonical jobs"/"raw jobs stored"/"CVs on file"	Відповідають реальним даним на Jobs/Sources/Profile	P2

Profile / CV (CV)

ID	Кроки	Очікуваний результат	Пріор.
CV-01	Завантажити .txt CV	З'являється у списку: ім'я файлу, дата, прев'ю тексту	P1
CV-02	Завантажити .pdf і .docx CV	Обидва формати успішно приймаються й розпізнаються	P1

CV-03	Завантажити файл невідтримуваного формату (.png)	Помилка 422 у банері, файл не додано	P2
CV-04	Натиснути "Analyze most recent CV"	Після завершення — картка CandidateProfile (досвід, ролі, навички з рівнем, досягнення, "Analyzed using ...")	P1
CV-05	Завантажити новий CV, повторно проаналізувати	Профіль оновлюється новими даними	P2
CV-06	Перевірити стан без жодного CV	Кнопка "Analyze" не показується, поки не завантажено CV	P2

Settings — Preferences (PREF)

ID	Кроки	Очікуваний результат	Пріор.
PREF-01	Заповнити всі поля переваг і зберегти	"Saved.", дані персистентні після перезавантаження сторінки	P1
PREF-02	Залишити частину полів пустими й зберегти	Приймається без помилок (усі поля опціональні)	P2
PREF-03	Ввести список через кому з зайвими пробілами ("React, Vue ,Node")	Розбивається на очищені елементи без порожніх значень	P3

Telegram (TG)

ID	Кроки	Очікуваний результат	Пріор.
TG-01	Відкрити Settings без підключеного бота	Показано username бота (посилання) або "No Telegram bot is configured...", якщо на сервері не задано токен	P2
TG-02	Ввести валідний chat_id → Connect	Статус змінюється на "Connected", показано bot_username	P1
TG-03	Ввести невалідний chat_id → Connect	Помилка 422 "invalid bot token"	P2
TG-04	"Send test message" після підключення	Успіх в UI + реальне тестове повідомлення (87%) приходить у Telegram	P1
TG-05	"Send test message" до підключення	422 "no Telegram bot connected"	P2
TG-06	Дочекатись реального матчу ≥85% поза тихими годинами	Повідомлення приходить один раз з коректним форматом (емодзі/%, назва+компанія, зарплата/локація/досвід, strengths/gaps, 2 оцінки, 4 inline-кнопки)	P1
TG-07	Повторний скоринг того самого матчу	Дубль сповіщення НЕ надсилається (ідемпотентність)	P2
TG-08	Натиснути inline-кнопку в Telegram-повідомленні	Нічого не відбувається на сервері — очікувано, НЕ баг (розділ 8)	P2

TG-09	Матч з оцінкою 65–74% або 75–84% без сильного salary/location-збігу	Повідомлення не надсилається ні одразу, ні пізніше — очікувано, НЕ баг	P3
TG-10	Матч $\geq 85\%$, що потрапляє у вікно тихих годин (22:00–08:00, час сервера)	Повідомлення не надсилається і не буде надіслано пізніше — очікувано, НЕ баг	P3

Sources (SRC)

ID	Кроки	Очікуваний результат	Пріор.
SRC-01	Відкрити Sources	Список dou/djinni з кількістю raw_jobs_stored	P1
SRC-02	"Sync now" для джерела	"Queuing..." → повідомлення про постановку в чергу; лічильник з часом росте	P1

Jobs — список (JOB)

ID	Кроки	Очікуваний результат	Пріор.
JOB-01	Відкрити список Jobs	Заголовок, компанія, бейдж practical_fit% (якщо оцінено), бейдж кількості джерел (якщо >1)	P1
JOB-02	Гортати сторінки (Previous/Next) на межах списку	Кнопки коректно вимикаються, лічильник "X–Y of Z" вірний	P1
JOB-03	Перемкнути "Show all, including low matches"	Список розширюється, включно з SKIP-вакансіями; offset скидається на 0	P2
JOB-04	Натиснути "Score unscored on this page" (за наявності неоцінених)	Усі ставляться в чергу; після оновлення сторінки з'являються бейджі оцінки	P2
JOB-05	Відкрити список на щойно розгорнутому застосунку (без вакансій)	"No jobs yet — trigger a sync..."	P2

Job Details (JD)

ID	Кроки	Очікуваний результат	Пріор.
JD-01	Відкрити неоцінену вакансію → "Score this job"	"Queued...", за кілька секунд + ручне оновлення з'являється оцінка	P1
JD-02	Відкрити оцінену вакансію	Practical fit, Requirement match, бейдж Recommendation, розбивка по 8 компонентах, strengths/gaps, "Skills identified using ..."	P1
JD-03	Натиснути "Rescore"	Вакансія повторно ставиться в чергу на переоцінку	P2
JD-04	Відкрити APPLY-вакансію	Картка "Should I apply?" показана АБО обґрунтовано відсутня (ліміт LLM-викликів вичерпано)	P2
JD-05	Коли картка "Should I apply?" показана	recommendation, confidence%, interview risk, summary, critical gaps, transferable experience, "Assessed using ..."	P2

JD-06	Оглянути сторінку на наявність дій Save/Apply/Reject/Hide	Їх немає — очікувано, НЕ баг	P3
-------	--	------------------------------	----

Заглушки (STUB)

ID	Кроки	Очікуваний результат	Пріор.
STUB-01	Прямий перехід на /applications	Лише заголовок "Applications", без даних/дій, без крашу	P3
STUB-02	Прямий перехід на /market-insights	Лише заголовок "Market Insights", без крашу	P3

12. Формат баг-репорту

Мінімальний набір інформації для кожного репорту, щоб розробник міг відтворити без уточнюючих запитань:

- **Заголовок** — коротко, у форматі "[Екран] що не так".
- **Кроки відтворення** — пронумеровано, з нуля (напр. "1. Залогінитись як ... 2. Відкрити /jobs ...").
- **Очікуваний результат** — з посиланням на розділ цього документа, якщо є (напр. "п. 5.7").
- **Фактичний результат** — що сталося насправді.
- **Скріншот / відео** — обов'язково для UI-багів.
- **Оточення** — браузер, локально/staging, час (для нотифікацій — важливо через тихі години).
- **Severity** — Blocker / Critical / Major / Minor / Trivial.

Перед тим як заводити тикет — звірте поведінку з розділами 8 і 9: якщо це очікуване обмеження поточної версії, тикет не потрібен.

13. Додаток: документація в репозиторії

Для глибшого занурення (розробницький рівень деталізації, не обов'язково для QA):

Файл	Зміст
README.md	Загальний огляд + статус фаз реалізації
ARCHITECTURE.md	Місія, принципи, стек, карта модулів
docs/domain-model.md	Сутності та їхні зв'язки
docs/source-adapters.md	Контракт адаптера джерела, ротація категорій, retention
docs/matching-engine.md	Повний конвеєр скорингу, ваги, LLM reranking
docs/notifications.md	Канали доставки, політика, формат повідомлення
docs/api.md	Поверхня REST API
docs/roadmap.md	Фази побудови й v1 definition of done
docs/deployment.md	Розгортання на Cloudflare Pages + Oracle Cloud

Документ згенеровано на основі прямого огляду коду репозиторію job-scraper (гілка main, коміт 0f0a858, 2026-08-31). У разі розбіжності з фактичною поведінкою застосунку — код є остаточним джерелом істини, цей документ варто оновити.